

LAB ASSIGNMENT 2

Static Application Security Testing (SAST) using Flawfinder

Submitted By:

Name: Nice Dhillon

Roll No: 2024UCP1367

Group: 15

Partner: Siddhi Aggarwal

Course: Cryptography Lab

1. Aim

To gain hands-on experience with a Static Application Security Testing (SAST) tool by installing **Flawfinder**, creating a vulnerable C program, performing static security analysis, identifying insecure coding practices, and documenting the complete analysis process.

2. Objective

- Learn about Static Application Security Testing.
- Install and configure Flawfinder.
- Create a vulnerable C program.
- Analyze the program using Flawfinder.
- Understand the detected vulnerabilities.
- Document the installation process and findings

3. Introduction to SAST

Static Application Security Testing (SAST) is a software security testing technique that analyzes the source code of a program without executing it. It helps developers identify security vulnerabilities during the development phase, allowing issues to be fixed before deployment. Since the application is not executed, SAST is capable of finding insecure coding practices,

unsafe function calls, hardcoded credentials, and many other security weaknesses early in the software development lifecycle.

4. Flawfinder

Flawfinder is an open-source Static Application Security Testing (SAST) tool designed specifically for C and C++ programs. It scans source code and identifies the use of insecure library functions that may introduce security vulnerabilities such as buffer overflows or command injection. Flawfinder performs static analysis, meaning it examines the source code without executing it. Each detected issue is assigned a **risk level from 0 to 5**, where higher values indicate more severe security risks.

6. Installation

```
[azureuser@Ubuntu:~/CryptoLab/crypto-lab/lab2$ sudo apt update
Hit:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Get:2 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease [126 kB]
Get:3 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease [126 kB]
Get:4 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease [126 kB]
Get:5 http://ports.ubuntu.com/ubuntu-ports noble-updates/main arm64 Packages [1225 kB]
Get:6 http://ports.ubuntu.com/ubuntu-ports noble-updates/main arm64 Components [178 kB]
Get:7 http://ports.ubuntu.com/ubuntu-ports noble-updates/universe arm64 Packages [1708 kB]
Get:8 http://ports.ubuntu.com/ubuntu-ports noble-updates/universe arm64 Components [387 kB]
Get:9 http://ports.ubuntu.com/ubuntu-ports noble-backports/main arm64 Components [3568 B]
Get:10 http://ports.ubuntu.com/ubuntu-ports noble-backports/universe arm64 Components [12.6 kB]
Get:11 http://ports.ubuntu.com/ubuntu-ports noble-security/main arm64 Components [43.2 kB]
Get:12 http://ports.ubuntu.com/ubuntu-ports noble-security/universe arm64 Components [76.3 kB]
Fetched 4012 kB in 4s (933 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
[azureuser@Ubuntu:~/CryptoLab/crypto-lab/lab2$ sudo apt install flawfinder
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  flawfinder
0 upgraded, 1 newly installed, 0 to remove and 4 not upgraded.
Need to get 60.5 kB of archives.
After this operation, 197 kB of additional disk space will be used.
Get:1 http://ports.ubuntu.com/ubuntu-ports noble/universe arm64 flawfinder all 2.0.19-1.1 [60.5 kB]
Fetched 60.5 kB in 1s (52.4 kB/s)
Selecting previously unselected package flawfinder.
(Reading database ... 109588 files and directories currently installed.)
Preparing to unpack .../flawfinder_2.0.19-1.1_all.deb ...
Unpacking flawfinder (2.0.19-1.1) ...
Setting up flawfinder (2.0.19-1.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...

Scanning linux images...
]
Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

7. Vulnerable Test Program

code.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char name[20];
    char buffer[20];
    char cmd[100];

    printf("Enter your name: ");

    gets(name);

    strcpy(buffer, name);

    sprintf(cmd, "echo Hello %s", buffer);

    system(cmd);

    return 0;
}
```

8. Static Analysis

GNU nano 7.2

flawfinder version 2.0.19, (C) 2001-2019 David A. Wheeler.

Number of rules (primarily dangerous function names) in C/C++ ruleset: 222

Examining code.c

FINAL RESULTS:

code.c:12: [5] (buffer) gets:

Does not check for buffer overflows (CWE-120, CWE-20). Use fgets() instead.

code.c:14: [4] (buffer) strcpy:

Does not check for buffer overflows when copying to destination [MS-banned] (CWE-120). Consider using snprintf, strcpy_s, or strncpy (warning: strncpy easily misused).

code.c:16: [4] (buffer) sprintf:

Does not check for buffer overflows (CWE-120). Use sprintf_s, snprintf, or vsnprintf.

code.c:18: [4] (shell) system:

This causes a new program to execute and is difficult to use safely (CWE-78). try using a library call that implements the same functionality if available.

code.c:6: [2] (buffer) char:

Statically-sized arrays can be improperly restricted, leading to potential overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use functions that limit length, or ensure that the size is larger than the maximum possible length.

code.c:7: [2] (buffer) char:

Statically-sized arrays can be improperly restricted, leading to potential overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use functions that limit length, or ensure that the size is larger than the maximum possible length.

code.c:8: [2] (buffer) char:

Statically-sized arrays can be improperly restricted, leading to potential overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use functions that limit length, or ensure that the size is larger than the maximum possible length.

ANALYSIS SUMMARY:

Hits = 7

Lines analyzed = 21 in approximately 0.00 seconds (7667 lines/second)

Physical Source Lines of Code (SLOC) = 14

Hits@level = [0] 1 [1] 0 [2] 3 [3] 0 [4] 3 [5] 1

Hits@level+ = [0+] 8 [1+] 7 [2+] 7 [3+] 4 [4+] 4 [5+] 1

Hits/KSLOC@level+ = [0+] 571.429 [1+] 500 [2+] 500 [3+] 285.714 [4+] 285.714 [5+] 71.4286

Minimum risk level = 1

Not every hit is necessarily a security vulnerability.

You can inhibit a report by adding a comment in this form:

// flawfinder: ignore

Make **sure** it's a false positive!

You can use the option `--neverignore` to show these.

There may be other security vulnerabilities; review your code!

See 'Secure Programming HOWTO'

(<https://dwheeler.com/secure-programs>) for more information.

9. Vulnerabilities Detected

Function	Risk	Description
gets()	5	Reads input without checking buffer size, causing possible buffer overflow.
strcpy()	4	Copies strings without validating destination buffer size.
sprintf()	4	May overflow the destination buffer if input is large.
system()	4	Executes operating system commands and may allow command injection if user input is used.

10. Analysis of Findings

1. gets()

Flawfinder reported gets() because it reads user input without checking the buffer size. If a user enters more characters than the buffer can hold, a buffer overflow may occur, leading to memory corruption or arbitrary code execution.

2. strcpy()

The `strcpy()` function copies the entire source string into the destination buffer without verifying its size. If the source string is larger than the destination buffer, memory outside the allocated space can be overwritten.

3. `sprintf()`

The `sprintf()` function writes formatted output into a character array but performs no bounds checking. If the resulting string exceeds the size of the destination buffer, it may overwrite adjacent memory locations.

4. `system()`

The `system()` function executes shell commands. If the command string contains user-controlled input, an attacker may execute unintended or malicious commands on the system.

11. Advantages of Flawfinder

- Easy to install and use.
- Fast analysis of C/C++ source code.
- Detects common insecure library functions.
- Provides risk levels for prioritizing vulnerabilities.
- Useful during software development before deployment.

12. Limitations

- Works only for C and C++ programs.
- Cannot detect runtime vulnerabilities.
- May produce false positives.
- Does not automatically fix detected issues.

13. Conclusion

In this experiment, Flawfinder was successfully installed and used to analyze a vulnerable C program. The tool detected several insecure functions, including `gets()`, `strcpy()`, `sprintf()`, and `system()`, highlighting common security issues such as buffer overflow and command injection. The experiment demonstrated how Static Application Security Testing helps developers identify vulnerabilities early in the development process, improving the overall security and reliability of software.